

Experimento 1 - Título

1. Identificação do Grupo

Integrante 1: Douglas Monteiro Almeida Souza

NUSP 1: 10748048

Integrante 2: Guilherme Junji Tutiya

NUSP 2: 14576065

Integrante 3: Henrique Maruiti

NUSP 3: 12610243

<https://github.com/Douglas-At/PCS3732-laboratorio-processadores>

2. Leitura Pré-Aula

Apesar de ter lido o guia resumido (alias, o arquivo no modle [Guia Resumido do Arduino IDE 2.3.8](#), ao abrir o arquivo o título esta como Guia Resunido do Arduino IDE 2.3.8, está com um pequeno typo) ainda tivemos algumas dificuldades na etapa de instalação do suporte pela Espressif

3. Especificação de Requisitos e Testes

Especificação de requisitos funcionais (RF) e **não-funcionais** (RNF, vide norma [ISO25010](#)) que o circuito a ser projetado deve atender e os **testes** associados a cada requisito mapeado, conforme especificado na Tabela 1.

Requisito	RF ou RNF	Teste	Resultado Esperado	Resultado Obtido	Evidências de Resultados
Breve descrição do requisito	Especificar se é requisito funcional ou não-funcional	Breve descrição do caso de teste	Descrever o resultado esperado	Para cada resultado apresentado, discutir se ele está adequado ou não.	Citar capturas de tela, fotos e outras evidências que podem ser inseridas após
Piscar LED	Não funcional	Testar a funcionalidade da placa ESP32	O LED deve alternar a cada intervalo de 1 segundo	O LED alternou entre aceso e apagado a cada 1 segundo	O LED acende e desliga alternadamente. Vídeo
Semáforo	Funcional	Verificar a	O LED deve	O semáforo	O semáforo

		transição de cores do LED da placa ESP32	manter a cor verde por 3 segundos, a cor amarelo por 1 segundo e a cor vermelha por 4 segundos, necessariamente nessa ordem	funcionou como esperado	funcionou como esperado Vídeo
--	--	--	---	-------------------------	--

Tabela 1 - Requisitos e Testes Planejados

4. Arquitetura do Sistema

Por ser apenas a primeira aula a arquitetura é muito simples, foi feito o desenvolvimento no computador no Arduino IDE, e depois usamos a ESP32 C3 MODULE, e funcionou,

5. Método Experimental

Usamos os seguintes recursos:

Arduino IDE 2.x: Ambiente de desenvolvimento, compilação e upload do firmware

Espressif ESP32 Board Package: Suporte à placa ESP32-C3 no Arduino IDE

ChatGPT: Análise do arquivo .elf e identificação de instruções Assembly

GitHub: Controle de versão e repositório público dos códigos

<https://github.com/Douglas-At/PCS3732-laboratorio-processadores>

Câmera (smartphone): Registro de vídeos e fotos das evidências experimentais

e para a execução do trabalho seguimos com as seguintes etapas:

Etapa 1: Instalação e Configuração

Instalar o Arduino IDE, adicionar o suporte ao ESP32 via Board Manager e selecionar a placa ESP32C3 Dev Module

Etapa 2: Experimento Blink

Carregar o exemplo Blink, testar delays de 1000, 500, 250 e 100 ms e registrar se digitalWrite(LED_BUILTIN, HIGH) acende ou apaga o LED na placa utilizada. (video anexado na tabela 1)

Etapa 3: Exportação e Análise ELF

Exportar o binário compilado (.elf), submetê-lo ao Gemini e identificar as instruções Assembly equivalentes a pinMode() e digitalWrite(), comparando com o Blink.ino.

Etapa 4: Semáforo Completo

Implementar o ciclo verde (3 s) → amarelo (1 s) → vermelho (4 s), cronometrar 3 ciclos completos e registrar em vídeo como evidência do RF03.(video anexado na tabela 1)

Etapa 5: Brincamos com as possibilidades de cores

depois que entendemos como funcionava o funcionamento das cores do led, fizemos um gradiente de cores. 📺 20260514_153037.mp4

6. Lições Aprendidas

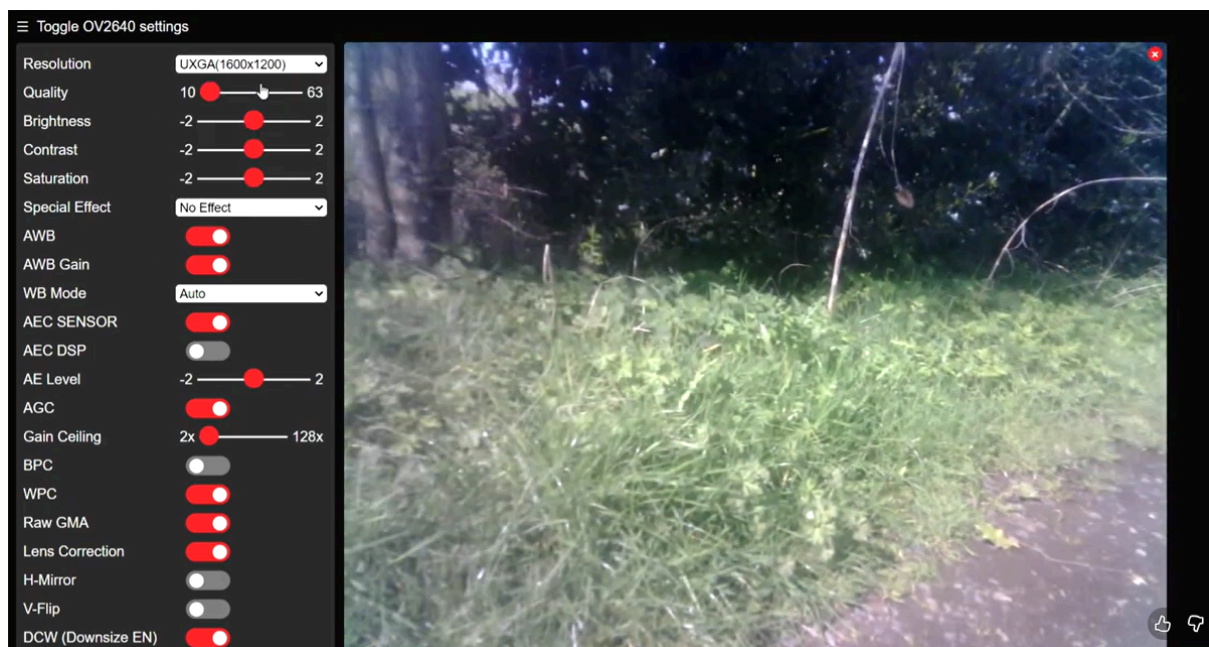
Felizmente 2 dos 3 integrantes tinham já conhecimento previo sobre o uso de ESP, então o aprendizado foi mais relembrar como funcionava o uso geral.

Outro aprendizado foi a partir da análise do arquivo .elf e equivalência com Assembly RISC-V:

A análise via GPT mostrou que `pinMode()` é compilado como um tail call direto passando o pino (GPIO 30) e o modo em registradores `a0/a1`, enquanto `digitalWrite()` gera um caminho otimizado que chama diretamente `gpio_set_level` na ROM do chip. No ESP32-C6, `digitalWrite(LED_BUILTIN, HIGH)` acende o LED — lógica ativa alta, oposta à de placas com LED invertido.

7. Projeto

O projeto consiste em um sistema de segurança que simula o funcionamento de um porteiro eletrônico. A ideia central é utilizar um módulo com câmera — ESP32-CAM ou câmera do Raspberry Pi, conforme orientação do professor — integrado a um computador central, que recebe o stream de vídeo e exibe uma interface de monitoramento em tempo real



<https://www.youtube.com/watch?v=YG08SI1JbQw>

Um botão físico do kit funciona como acionador de alerta: ao identificar algo suspeito, o segurança o pressiona e o sistema envia uma notificação de emergência. A referência técnica principal para a implementação com ESP32-CAM é o tutorial, que demonstra a

configuração do streaming via Wi-Fi. A versão com Raspberry Pi seguirá arquitetura equivalente, substituindo o módulo ESP32-CAM pela câmera dedicada da plataforma.